

BACKEND(SUNDAY 08/03/2026):

0001_initial.py: This Django migration file creates the database tables for reservations and ratings in our carpooling app and for tracking trip bookings with prices statuses and timestamps and user ratings with detailed scores and reviews in your carpooling app

Table 1: **reservations** - Booking Information

Stores all trip booking made by passengers

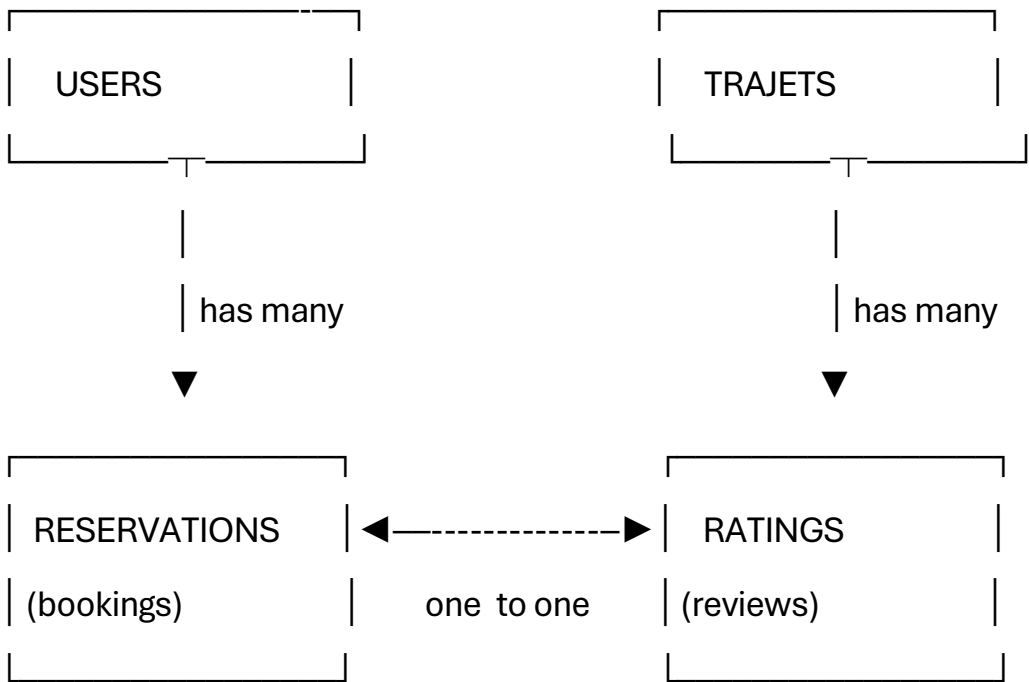
Field	Purpose	Example
nbr_places	How many seats booked	2 seats
status	Current booking state	"CONFIRMED"
price_per_seat	Price when booked	500 DA
total_price	Total cost	1000 DA
passager	Who booked	User ID 42
trajet	Which trip	Trip ID 123

Table 2: **ratings** - User Reviews

Stores ratings and reviews between users after trips

Field	Purpose	Example
note	Overall rating (1-5)	5 stars
comment	Written review	"Great driver!"
ponctualite	Punctuality score	5
convivialite	Friendliness score	4

conduite	Driving score (for drivers)	5
rated	Who receives rating	User ID 42
rater	Who gives rating	User ID 15



Admin.py: this code configures the Django admin panel for managing reservations and ratings through a nice web interface

For **ReservationAdmin** - Managing Bookings

Creates an admin interface to view filter and manage all trip bookings

```
# In Django admin, you'll see:
=====
| ID | Trajet                | Passager | Places | Statut | Prix |
=====
| 42 | Alger→Oran (2024-03-15) | John     | 2      | ✓      | 1000 |
| 43 | Oran→Constantine (2024-03-16) | Sarah   | 1      | ⌚      | 500  |
=====
```

For **RatingAdmin** - Managing Reviews

Create an admin interface to view all user rating and reviews

In Django admin, you'll see:

```
=====
| Rater | Rated | Note | Stars | Date |
=====
| John  | Karim | 5    | ★★★★★ | 2024-03-15 |
| Sarah | Ahmed | 4    | ★★★★☆ | 2024-03-16 |
=====
```

And all this makes a simple admin list to display all information

```
list_display = [
    "id",          # Booking ID number
    "trajet_info", # Custom: "Alger→Oran (2024-03-15)"
    "passager",    # Who booked
    "nbr_places",  # How many seats
    "status_badge", # Custom: Colored status
    "total_price", # Total cost
    "created_at",  # When booked
]
```

```
def status_badge(self, obj):
    colors = {
        "PENDING": "orange", # ⌚ Waiting
        "CONFIRMED": "green", # ✅ Approved
        "REJECTED": "red",    # ❌ Refused
        "CANCELLED": "gray",  # ✖ Cancelled
    }
    # Shows: PENDING in orange, CONFIRMED in green, etc.
```

```
def stars_display(self, obj):
    stars = "★" * obj.note # 5 stars = "★★★★★"
    return format_html('<span style="font-size: 16px;">{}</span>', stars)
```

```
fieldsets = (
    ("Réservation", {"fields": ("trajet", "passager", "nbr_places")}),
    ("Tarification", {"fields": ("price_per_seat", "total_price")}),
    ("Statut", {"fields": ("status", "rejection_reason", "cancellation_reason")}),
    ("Dates", {"fields": ("created_at", "approved_at", "cancelled_at")}),
)
# Groups related fields together in the edit form
```

Action	How
--------	-----

View all bookings	List view with all details
Filter by status	Click PENDING/CONFIRMED/etc.
Search users/trips	Type in search box
Edit bookings	Click ID to open form
Change status	Dropdown to update
Add reasons	Text fields for rejections
See history	Date stamps for all actions
View ratings	Stars display for quick overview

Apps.py: the file configuration the reservations app in Django and loads its signal handlers when the app starts and automatic notifications work whenever users book trips or leave ratings

1.BASIC APP CONFIGURATION :

```
class ReservationsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "app.reservations"
    verbose_name = "Réservations"
```

Name tells Django where this app is located and verbose_name for human readable name shown in admin reservations and default_auto_field and uses 64-bit integers for primary keys

2.SIGNAL LOADING :

```
def ready(self):
    """Charge les signals au démarrage"""
    print("=" * 80)
    print("ReservationsConfig.ready() appelé")
    print("=" * 80)

    try:
        import app.reservations.signals # noqa: F401
```

```

        print("Signals réservations chargés avec succès")
    except Exception as e:
        print(f"Erreur chargement signals réservations: {e}")
        import traceback

        traceback.print_exc()

```

When Django starts it calls `ready()` and imports the `signals.py` file so that all the notification triggers are active

And with this configuration at startup signals are loaded and when someone books a trip the notification sent automatically

```

Django Server Starts
↓
Loads all apps in INSTALLED_APPS
↓
Finds "app.reservations"
↓
Calls ReservationsConfig.ready()
↓
Imports app.reservations.signals
↓

```

All booking signals are now ACTIVE - New booking → notify driver - New rating → notify user - Cancellation → notify passengers

Apps.py: with this file (**configures the reservations app**) in Django and with it the notifications are loaded when the server starts directly

```

def ready(self):
    """Charge les signals au démarrage"""
    print("=" * 80)
    print("ReservationsConfig.ready() appelé")
    print("=" * 80)

    try:
        import app.reservations.signals # noqa: F401

        print("Signals réservations chargés avec succès")
    except Exception as e:
        print(f"Erreur chargement signals réservations: {e}")
        import traceback

        traceback.print_exc()

```

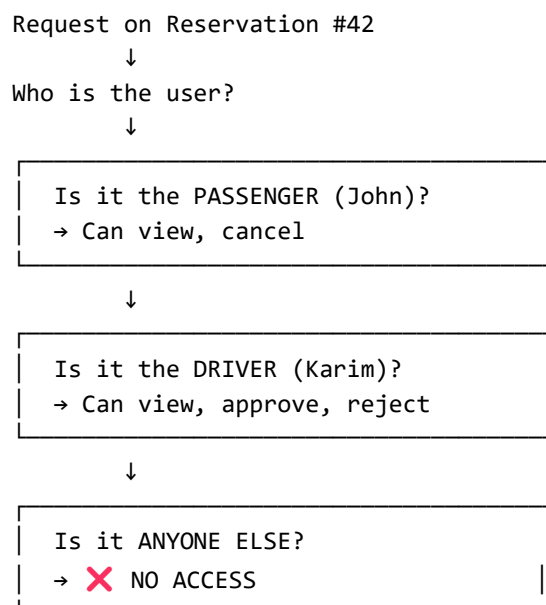
When Django starts it calls `ready()` and imports the signals file so that all notifications triggers become active and with this at startup signals are loaded and ready to respond to events and notifications work whenever users book trips or leave ratings

Django Starts
↓
Finds "app.reservations" in INSTALLED_APPS
↓
Calls `ReservationsConfig.ready()`
↓
Imports `app.reservations.signals`
↓
ALL SIGNALS NOW ACTIVE:

- New booking → notify driver
- New rating → notify rated user
- Trip cancelled → notify passengers

Models.py: this file defines the database structure for managing trip booking and user ratings in our carpooling app and with it all the information about the reservation of a trip will be loaded to our database directly and the request, approval, rejection, cancellation and handle user ratings with details criteria and automatic statistic updates (BDD DAIGRAMME)

Permissions.py: by this file our site can define who can do what with the reservations based on their role as passenger or driver



Action	Passenger	Driver	Anyone Else
View reservation	✓	✓	✗
Cancel booking	✓	✗	✗
Approve booking	✗	✓	✗
Reject booking	✗	✓	✗
Leave rating	✓	✓	✗

So this file ensures that the right people can do the right things with reservations passengers can cancel their own booking and drivers can approve/reject them and both can rate each other after the trip

Serializers.py: we can consider this file as the point of communication between python/Django and JSON for the API with special logic to handle ratings

```
class RatingSerializer(serializers.ModelSerializer):
    """Serializer pour les évaluations"""
# Example output:
{
    "id": 42,
    "note": 5,
    "comment": "Great ride!",
    "ponctualite": 5,
    "convivialite": 5,
    "rater_detail": {"id": 1, "email": "john@example.com"},
    "rated_detail": {"id": 2, "email": "karim@example.com"}
}
```

```
class ReservationSerializer(serializers.ModelSerializer):
    """
    Serializer CORRIGÉ pour afficher correctement can_rate et has_rating
    """
# Example output for a booking:
{
    "id": 123,
    "status": "CONFIRMED",
    "nbr_places": 2,
    "total_price": 1000,
    "can_rate": true,          # User CAN leave a rating
    "has_rating": false,      # No rating yet
}
```

```

    "rating": null,          # No rating data
    "passager_detail": {"email": "john@example.com"},
    "trajet_detail": {"from": "Alger", "to": "Oran"}
}

```

```

class ReservationCreateSerializer(serializers.ModelSerializer):
    """Serializer pour la création d'une réservation"""

```

This one is used to handles validation when a user wants to book a trip and checks performed so that: can't book your own trip, can't book more seats than available, can't book the same trip twice

```

class RatingCreateSerializer(serializers.Serializer):
    """Serializer simplifié pour la création de ratings"""

```

```

# What the frontend sends:
{
    "note": 5,
    "comment": "Great driver!",
    "ponctualite": 5,
    "convivialite": 5
}

```

When all works together:

```

Frontend requests booking #123
↓
ReservationSerializer runs
↓
Checks: Is user logged in?
        Is user the passenger?
        Is booking confirmed?
↓
Sets can_rate = true/false
Sets has_rating = true/false
Includes rating data if exists
↓
Frontend knows:
- "Show rating form" (can_rate = true)
- "Already rated" (has_rating = true)
- "Show stars" (rating data exists)

```

Signals.py: the file automatically triggers actions whenever a reservation is created, changed or deleted

Event	What Happens Automatically
-------	----------------------------

New booking (PENDING)	• Notifies driver • Reserves seats
Booking CONFIRMED	• Notifies passenger • Creates/updates group chat
Booking REJECTED	• Notifies passenger
Booking CANCELLED	• Notifies driver • Restores seats
Booking DELETED	• Restores seats

New booking __ notify driver & reserve seats :

When someone books:

```
Reservation.objects.create(passager=John, trajet=Trip#42)
```

AUTOMATICALLY:

🔔 Driver gets: "John wants to join your trip"

🚗 Available seats: 4 → 3 (seats reserved)

Booking confirmed __ create group chat :

When driver approves:

```
booking.approve() # status becomes CONFIRMED
```

AUTOMATICALLY:

🔔 Passenger gets: "Your booking is confirmed!"

💬 Group chat created with:

- Driver (Karim)

- All confirmed passengers (John, Sarah, etc.)

✉️ Welcome message sent

Booking cancelled __ restore seats :

When someone cancels:

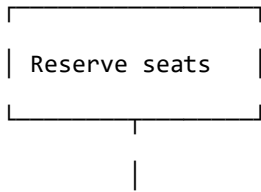
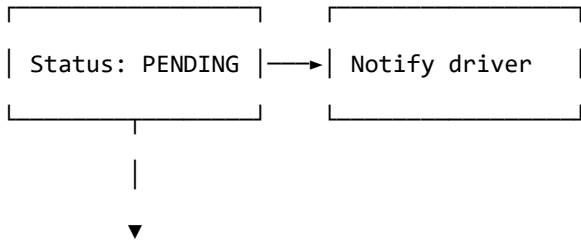
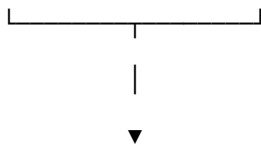
```
booking.cancel()
```

AUTOMATICALLY:

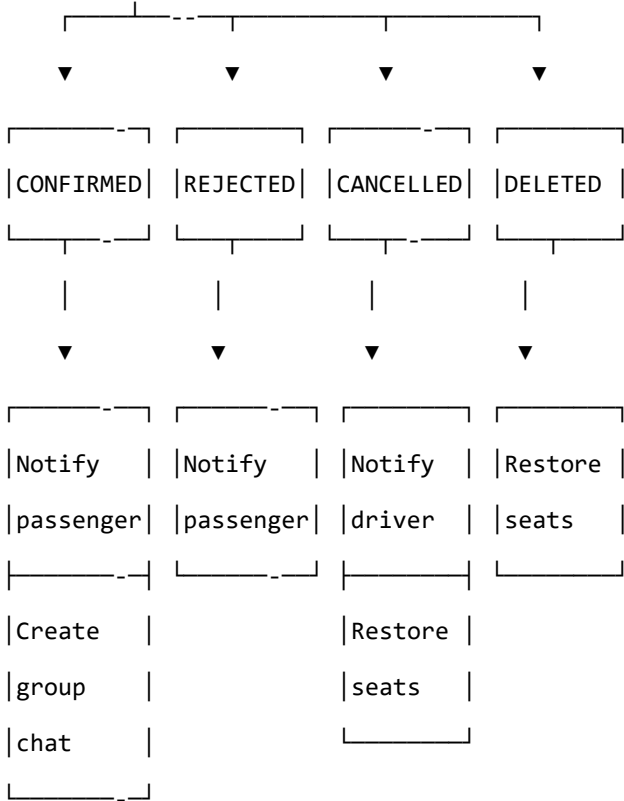
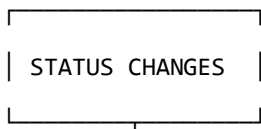
🔔 Driver notified: "John cancelled"

🚗 Available seats: 3 → 4 (seats freed up)

| NEW RESERVATION |



▼ (Driver action)



Urls.py: this file defines all the API endpoints for managing reservation in our site web

POST	/api/v1/reservations/	→ Create a new booking
GET	/api/v1/reservations/	→ List all bookings
GET	/api/v1/reservations/{id}/	→ View specific booking
PUT	/api/v1/reservations/{id}/	→ Update booking
DELETE	/api/v1/reservations/{id}/	→ Delete booking
GET	/api/v1/reservations/my-bookings/	→ Bookings I made as passenger
GET	/api/v1/reservations/my-trips-bookings/	→ Bookings on my trips as driver
POST	/api/v1/reservations/{id}/confirm/	→ Driver accepts booking
POST	/api/v1/reservations/{id}/reject/	→ Driver rejects booking
POST	/api/v1/reservations/{id}/cancel/	→ Passenger cancels booking
GET	/api/v1/reservations/check-booking-permission/	→ Check if user can book

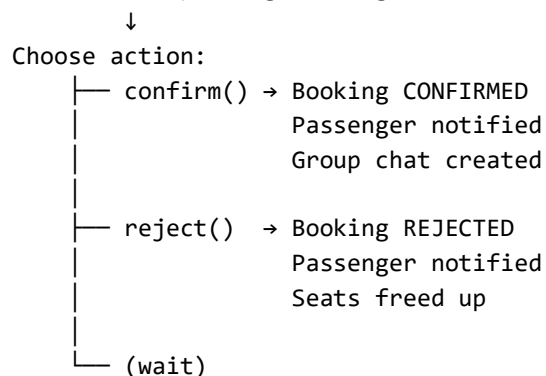
Views.py: this file handles all API requests for reservations and managing the entire booking lifecycle

Passenger creates booking:

1. User clicks "Book" on a trip
2. View checks: verified ID? ☒
3. View checks: seats available? ☒
4. View checks: not already booked? ☒
5. Creates PENDING reservation
6. Driver gets notified

Drive manages booking:

Driver sees pending booking



After trip – rating:

Trip completed

↓
Passenger can rate driver



View checks:

- Booking CONFIRMED? ☒
- Not already rated? ☒
- Valid rating (1-5)? ☒



Rating saved

Driver's average updates

This View Set manages the entire reservation process—from checking if users have verified documents before allowing bookings, handling confirmations/rejections/cancellations, and finally enabling ratings after trips.

SAIDI_MAHMOUD_2CP7